

千依

千依是一个解决方案协作网络，将问题的解决过程沉淀为可复用、可执行、可持续演进的资产。

从问题开始

企业真正的问题，往往不是缺少能力，而是两件事同时发生：**问题发生时，不知道从哪里开始；问题解决后，什么都没有留下。**

协作启动的代价

新员工入职第三天，需要申请一个系统权限。

他问了 HR，HR 说找 IT。IT 说要主管先审批。主管说已经批过了。IT 说没收到，要在内部系统里提单。内部系统的入口没人告诉他。他翻了半小时聊天记录，找到一个链接，打开是 404。

最后，他问了旁边的同事，同事花了十分钟带他走了一遍流程。

问题本身不复杂。真正消耗的，是找到那条路的时间。

这不是因为组织没有流程。而是当问题发生时，路只存在于某些人的记忆里，不在任何地方可以直接走进去。

遗忘的代价

又一个新员工入职，遇到了同样的权限问题。同样的问题，同样的兜转，同样的时间消耗。没有人觉得奇怪，因为这件事“本来就是这样的”。

千依要解决的，正是这两件事

启动成本和遗忘成本叠加在一起，产生一个结果：**组织永远在从零开始。**

问题发生时，需要重新找路。问题解决后，走过的路随即消失。下一次遇到同样的问题，重新沟通、重新协调、重新解决。

千依的目标是打破这个循环：让协作从问题发生的那一刻开始，并让每一次协作都成为下一次解决问题的基础。

两把刀，是千依打破这个循环的方式。

两把刀

第一刀：知道找谁

如果知道这件事该找谁负责，千依就能立刻开始工作。

问题发生 → 指定负责人 → 启动协作

不需要提前设计流程，不需要系统接入，不需要任何数字化建设。只要组织有基本的责任归属意识，第一刀就能落下去。

第二刀：知道要什么

现实中，很多人并不知道该找谁。但他们知道自己需要什么：我要报销，我要打印合同，我要采购材料，我要修一台设备。他们认知的不是组织架构，而是能力。

千依允许从能力出发：

描述需要什么能力 → 解析能力归属 → 定位负责人 → 启动协作

第二刀补上了第一刀覆盖不到的场景。当责任边界模糊时，能力往往仍然清晰。

路不是设计出来的，是走出来的

两把刀合在一起，意味着问题发生的那一刻就是协作开始的那一刻。路不需要提前铺好——解决问题的过程本身会把路踩出来，并留在千依里，供下一次直接走进去。

解决方案的三个演进阶段

每一次问题被解决，千依都会沉淀下这个过程，形成可复用的解决方案 **Flow**。

Flow 不是设计出来的。Flow 是一一次次解决问题之后长出来的。

Flow 不是预先设计好的流程图，而是组织解决某类问题能力的当前状态。它随着问题被反复解决而持续演进，存在三个阶段：

全 HUMAN：第一次遇到问题时，组织往往没有数字化能力。Flow 可以完全由人工节点组成，千依负责协调、通知与记录。这是所有解决方案的起点，无需任何数字化前提。

引入 Task：随着问题被反复解决，可重复、可标准化的环节自然浮现。这些环节被抽象为 Task，交给系统执行。人专注于判断，机器负责执行。

引入 Agent：当某些环节具备 AI 化条件，Agent 接管部分认知工作。人只在真正需要时介入。

没有人规定 Flow 必须到达哪个阶段。它停留在哪里，取决于组织当下拥有什么能力，以及认为哪些环节值得演进。数字化和 AI 化，是解决方案不断演进后的自然结果，而不是开始使用千依的前提。

两个角色，一个循环

千依围绕两个角色运转。

FDE (Flow Development Engineer) 是组织能力的建设者。他们将 ERP、MES、数据库、外部服务、物理设备、AI 服务封装为可调用的 Task，注册到千依中。FDE 不需要设计每一个解决方案，只需要持续扩大组织的能力边界。

业务人员 是解决方案的设计者。他们用现有的 Task 和人工节点组合成 Flow，描述解决某类问题的完整路径。在演进 Flow 的过程中，他们自然会发现哪些环节还缺少对应的 Task。

这形成一个循环：



组织能力在这个循环中持续生长。没有起点的门槛，也没有终点。

为什么不是 workflow 引擎

workflow 引擎解决的问题是：已知流程如何自动执行。它默认组织已经完成流程梳理、系统接入和数字化建设——它是数字化建设的结果，不是起点。

千依解决的问题是：当流程尚未存在时，协作如何立刻开始，解决方案如何从零生长。

workflow 引擎：流程 → 执行

千依：问题 → 协作 → 解决方案 → 能力 → 演进

组织不需要先拥有流程，才能开始使用千依。只要知道找谁，或者知道需要什么，千依就已经在工作了。

组织不需要先拥有解决方案。千依存在的意义，就是让解决方案自己生长出来。

快速开始